

Day06 Part A 学习文档 v1.0 : Memory Foundation (记忆基础模型)

本文是《从零实现 Agent Runtime》学习阶段的 Day06 Part A 正式学习文档。

Day05 已经完成 Execution Engine : Tool Calling、Tool Decision、Tool Schema、Tool Registry、Tool Executor、Permission、Observation Feedback、Multi Tool Loop 与 Mini Tool Runtime。Day06 开始进入 Memory System，回答一个新的问题：

Agent Runtime 已经能思考、能行动、能把工具结果写回当前状态，那么它如何跨会话、跨任务、跨时间地保留长期有价值的信息？

本节定位

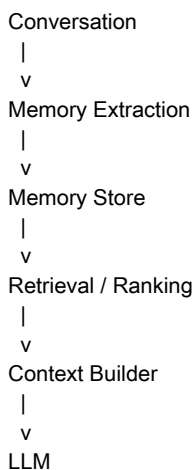
Day04 的 Runtime State 解决的是当前任务生命周期内的运行现场。

Day05 的 Tool System 解决的是 Agent 如何把结构化行动意图交给 Runtime 执行。

Day06 的 Memory System 解决的是：

Agent 如何从过去交互中筛选、沉淀、检索和使用长期信息。

Memory 不是“把聊天记录保存下来”，也不是“加一个向量数据库”。它是 Agent Runtime 的长期状态系统：



本节核心结论是：

Memory 保存的不是所有历史，而是未来可能影响 Agent 行为和决策的长期有效信息。Memory 是 Runtime 的长期 State，必须经过提取、分类、去重、更新、遗忘、检索和投影，才能真正进入 Agent Loop。

目录

Day06 是否需要拆分成多个 Part

什么是 Agent Memory

为什么 Agent 需要 Memory

Stateless Agent 与 Stateful Agent

Conversation 不等于 Memory

Memory Extractor

Memory Extractor 本质上也是一个小 Agent

Memory Extractor 是否需要 LLM

工业 Memory 的混合提取方案

Memory 不是只 Create

Memory 与人类记忆的类比

Memory 是 Runtime 的长期 State

Memory 也需要 Context Budget

Memory 需要 Confidence

Memory 是安全边界

Part A 最终模型

本 Part 核心知识点

写书 TODO

写书素材

本 Part 核心认知升级

工业级实现

知识地图

面试视角

本章思考题

前置问题回收

下一节学习计划

Day06 是否需要拆分成多个 Part

Day06 建议继续像 Day05 一样拆成多个 Part。

原因是 Memory System 看起来像一个“存储问题”，但真正展开后，它其实覆盖了 Agent Runtime 的多个关键边界：

- 什么值得被记住
- 记忆如何被抽取
- 记忆如何被分类
- 记忆如何存储
- 记忆如何去重、合并、更新和遗忘
- 记忆如何被检索
- 记忆如何进入 Context Builder
- 记忆如何受到权限、隐私和安全策略控制

如果把 Day06 压成一篇，就很容易把 Memory 简化成：

Memory = Chat History + Vector Database

这会漏掉工业系统中最重要的 Runtime 设计问题。

更合理的学习节奏是：

- Part A：Memory Foundation，建立 Memory 的基础模型。
- Part B：Memory Architecture，理解 Memory Store、Retriever、Ranker、Injector。
- Part C：Memory Lifecycle，理解 create、update、merge、forget、decay。
- 后续 Part 再继续展开 Retrieval、Context Injection、Safety 与 Mini Memory Runtime。

本节先完成 Part A：Memory 的基础认知。

什么是 Agent Memory

Agent Memory 是 Runtime

从历史交互、任务过程、工具观察、用户反馈和业务事件中提取出的长期有效信息。

它不是完整聊天记录，也不是原始日志，而是经过筛选后的结构化知识。

例如用户说：

以后回答问题请多解释原理，不要只给答案。

Conversation 记录的是这句话本身。

Memory 保存的应当是：

```
{
  "type": "preference",
  "content": "User prefers detailed explanations instead of answer-only responses",
  "confidence": 0.92
}
```

这条 Memory 在未来回答中会影响 Agent 的行为风格。

为什么 Agent 需要 Memory

没有 Memory 的 Agent 是无状态的。

```
User Input
|
v
Runtime
|
v
LLM
|
v
Response
```

每一次请求结束后，用户身份、长期偏好、历史背景和任务延续性都会丢失。

加入 Memory 之后：

```
Memory
|
v
Runtime State
|
v
Context Builder
|
```

v
LLM

Agent 能做到：

- 记住用户是谁
- 理解用户长期偏好
- 延续过去任务背景
- 根据历史经验调整后续决策

核心认知是：

Memory 让 Agent 从“一次性回答系统”变成“持续交互系统”。

Stateless Agent 与 Stateful Agent

Stateless Agent 每次请求都像第一次见到用户。

它可以利用当前 prompt 和当前上下文回答问题，但无法稳定地跨会话延续用户信息。

Stateful Agent 不只是拥有 Runtime State，还拥有长期 Memory。

Runtime State 解决的是当前任务。

Memory 解决的是长期用户生命周期。

```
State
|
+-- Runtime State
|  |
|  +-- 当前任务
|  +-- 当前 Step
|  +-- 当前 Tool Result / Observation
|
+-- Memory State
    |
    +-- 用户画像
    +-- 长期偏好
    +-- 重要历史事件
    +-- 可复用知识
```

这也解释了为什么 Memory 是 Runtime 的一部分，而不是 LLM 本身的能力。

LLM 只能在当前上下文窗口里“看到”信息；Runtime 才负责决定哪些长期信息应该进入这一次上下文。

Conversation 不等于 Memory

Conversation 是完整事件记录。

Memory 是从事件记录中提取出的、未来可能有价值的信息。

错误理解：

Memory = 所有聊天记录

正确理解：

```
Conversation
|
v
Memory Extraction
|
```

v
Useful Long-term Knowledge

例如：

今天北京天气不错。

这句话可以出现在 Conversation 中，但通常不应该成为长期 Memory。

如果保存成：

```
{  
  "type": "fact",  
  "content": "User thinks Beijing weather is good"  
}
```

几个月后 Agent 可能错误地说：

你之前喜欢北京天气。

这就是 Memory 污染。

所以 Memory 的关键不是保存，而是筛选。

Memory Extractor

Memory Extractor 是 Memory System 中负责判断“是否应该记住”的模块。

它面对 Conversation、已有 Memory、业务上下文和隐私策略，输出结构化决策：

```
Conversation  
|  
v  
Memory Extractor  
|  
+-- ignore  
|  
+-- create memory  
|  
+-- update memory  
|  
+-- merge memory  
|  
+-- forget memory
```

Memory Extractor 的职责包括：

- 判断一段信息是否有长期价值
- 判断它属于哪种 Memory 类型
- 判断它是新建、更新、合并还是遗忘
- 给出 confidence
- 过滤不应该保存的隐私或敏感信息

Memory Extractor 是避免 Memory 污染的第一道关口。

Memory Extractor 本质上也是一个小 Agent

Memory Extractor 也可以看成一个小 Agent。

它也有自己的：

- Goal：判断当前信息是否值得长期保存
- Context：当前对话、已有 Memory、用户配置、隐私策略
- Decision：ignore / create / update / merge / forget
- Action：写入、更新、合并或删除 Memory

例如输入：

用户：
以后回答问题请多解释原理，不要只给答案。

Memory Extractor 的 Context 可能是：

```
{
  "conversation": "以后回答问题请多解释原理",
  "existing_memory": []
}
```

它的 Decision 可能是：

```
{
  "action": "create",
  "type": "preference",
  "content": "User prefers detailed explanations",
  "confidence": 0.92
}
```

然后 Runtime 执行：

```
MemoryStore.save()
```

这个过程和 Agent Loop 很像，只是目标从“完成用户任务”变成了“管理长期记忆”。

Memory Extractor 是否需要 LLM

工业上常见三种方案。

第一种是 LLM Memory Extractor。

```
Conversation
|
v
LLM
|
v
Memory JSON
```

它的优点是理解能力强，可以识别隐含偏好、稳定身份和语义相近的信息。

它的缺点是成本高、延迟高，并且需要更严格的输出校验。

第二种是 Rule + LLM Hybrid。

```
Conversation
|
v
Rule Filter
|
v
LLM Extractor
|
v
Memory Store
```

规则先过滤明显无价值的信息，只把可能有长期价值的片段交给 LLM 判断。

常见触发线索包括：

- 以后
- 永远
- 习惯
- 喜欢
- 不喜欢
- 我的工作
- 我主要使用

这种方案更接近工业系统，因为它兼顾成本、质量和可控性。

第三种是 Embedding + Similarity。

它主要用于判断新信息和已有 Memory 是否重复、相近或应该合并。

例如已有 Memory：

```
{
  "content": "User prefers TypeScript"
}
```

用户又说：

我更喜欢 TS

系统可以通过 embedding 相似度发现这两条信息高度相似，然后选择 update 或 merge，而不是重复 create。

工业 Memory 的混合提取方案

工业级 Memory 通常不是单一技术，而是多个步骤组合：

```
Conversation
|
v
Extraction
|
v
Classification
|
v
Deduplication
|
v
Storage
|
v
Retrieval
```

更完整一些可以写成：

```
Memory System
|
+-- Extraction
|
+-- Classification
|
+-- Deduplication
|
+-- Storage
|
```

```
+-- Retrieval
|
+-- Ranking
|
+-- Injection
|
+-- Lifecycle Management
```

所以 Memory System 不是一张数据库表，也不是一个向量库，而是一个 Runtime 子系统。

Memory 不是只 Create

很多初学者会把 Memory 理解为：

```
发现信息
|
v
保存
```

这只覆盖了 create。

工业 Memory 至少需要四类操作。

Create：第一次发现有长期价值的信息。

```
{
  "type": "preference",
  "value": "typescript"
}
```

Update：已有 Memory 发生变化或被补充。

```
{
  "old": "User uses TypeScript",
  "new": "User uses TypeScript and Rust"
}
```

Merge：多条相近 Memory 合并成更高层的信息。

```
{
  "content": "Frontend ecosystem preference: React + Next.js"
}
```

Forget：旧信息过期、被用户否定或不再可靠时，需要删除或降低权重。

这也引出 Memory Decay：

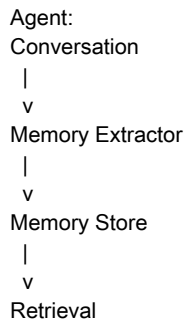
Memory 不是永久同权重存在的事实，而是会随着时间、证据和上下文变化而衰减或更新的长期状态。

Memory 与人类记忆的类比

Memory System 的设计很像人类记忆。

```
人类：
Experience
|
v
Attention / Filtering
|
v
Long-term Memory
|
v
Recall
```


对应到 Agent :



对应关系 :

| 人类 | Agent || --- | --- || 经历 | Conversation || 注意力 / 筛选 | Memory Extractor || 长期记忆 | Memory || 回忆 | Retrieval || 遗忘 | Decay / Forget Policy |

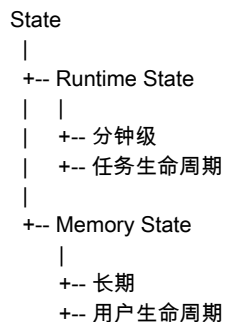
这个类比帮助理解一件事 :

记忆不是记录所有经历，而是筛选、抽象、保留、回忆和遗忘。

Memory 是 Runtime 的长期 State

Day04 讲的 Runtime State，是当前任务现场。

Memory 则是长期状态。



一个类比 :

Runtime State = 当前页面 JS state
Memory = localStorage

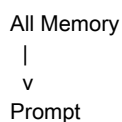
Runtime State 可以帮助 Agent 在一个任务中持续推进。

Memory 可以帮助 Agent 在多个任务之间形成连续性。

Memory 也需要 Context Budget

Memory 不能全部塞给 LLM。

错误方式 :



如果用户积累了一万条 Memory，全部注入上下文会直接造成 Prompt 爆炸。

所以 Memory Retrieval 必须回答 :

当前任务真正需要哪些 Memory ？

例如用户问：

帮我设计 React 架构。

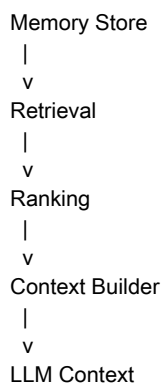
可能需要：

- Frontend background
- React preference
- Engineering style

不需要：

- 喜欢吃火锅
- 去年去哪里旅游
- 某次闲聊中的临时情绪

因此 Memory 最终也要进入 Context Builder 的预算管理：



Memory 需要 Confidence

不是所有 Memory 的可信度都一样。

例如用户说：

我最近可能考虑学习 Go。

这条信息带有不确定性，confidence 应该较低。

用户说：

我的主要技术栈是 React 和 Node。

这条信息更稳定，confidence 应该较高。

Memory 记录中应当包含质量信号：

```
{
  "content": "User uses React",
  "confidence": 0.95,
  "source": "conversation",
  "updatedAt": "2026-08-11"
}
```

Confidence 会影响：

- 是否保存

- 是否参与检索
- 是否进入上下文
- 是否覆盖旧 Memory
- 是否触发用户确认

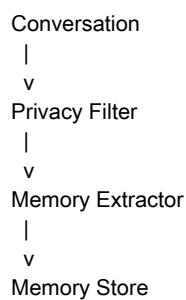
Memory 是安全边界

Memory System 必须考虑哪些内容不能保存。

例如：

- 密码
- Token
- 身份证号
- 支付信息
- 过度敏感的个人信息

因此 Memory Extractor 之前或内部需要 Privacy Filter。



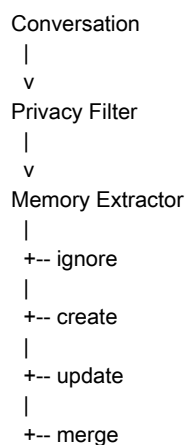
Memory 一旦写入长期存储，就可能跨会话、跨任务影响未来行为，所以它本身就是安全边界。

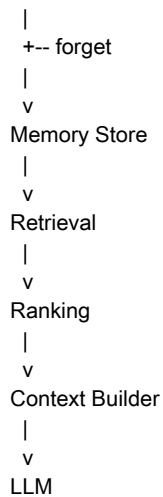
这与 Day05 的 Tool Permission 类似：

- Tool Permission 限制 Agent 能代表用户做什么。
- Memory Policy 限制 Agent 能长期记住什么。

Part A 最终模型

Day06 Part A 的 Memory 基础模型可以收束为：





这条链路说明：

- Memory 来源于 Conversation，但不等于 Conversation。
- Memory Store 只是 Memory System 的一个环节。
- Vector Database 只是 Memory Store 的一种实现。
- Memory 必须通过 Retrieval 和 Ranking 进入 Context Builder。
- Memory 需要生命周期管理和安全策略。

本 Part 核心知识点

本 Part 需要掌握：

- Memory 的定义
- 为什么 Agent 需要 Memory
- Stateless Agent vs Stateful Agent
- Conversation vs Memory
- Memory 的基础分类
- Memory Extractor 的作用
- Memory Extractor 为什么可以由 LLM 或规则 + LLM 驱动
- Memory create / update / merge / forget
- Memory Decay
- Memory 不是 Vector Database
- Memory 与 Runtime State 的区别
- Memory 与 Context Builder 的关系
- Memory 的 confidence 与安全边界

写书 TODO

未来写正式书稿时，需要把本 Part 整理为 Day06 的开篇章节。

重点补充：

- LLM 本身没有持续记忆能力
- Runtime 通过 Memory 扩展 Agent 的长期能力
- Memory 是 Context Builder 的输入源
- Memory 需要先筛选再保存
- Memory 污染会导致 Agent 长期误判
- Memory 与 Tool 一样，都是 Runtime 扩展 Agent 能力的关键子系统

可以形成一个基础论断：

Tool 解决 Agent 能做什么，Memory 解决 Agent 长期知道什么，Runtime 解决 Agent 如何组织思考、行动和状态。

写书素材

案例 1：Conversation 不等于 Memory。

用户：
我今天吃火锅。

Conversation：
User said they ate hotpot today.

Memory：
通常不保存。

用户：
我以后希望回答问题多解释原理。

Memory：
User prefers detailed explanations.

案例 2：Memory 类似人类记忆。

| 人类 | Agent | | --- | --- | | 经历 | Conversation | | 筛选 | Memory Extractor | | 长期记忆 | Memory | | 回忆 | Retrieval | | 遗忘 | Forget Policy |

案例 3：Memory 污染。

如果把“今天北京天气不错”保存成“用户喜欢北京天气”，未来 Agent 就可能产生错误个性化。

这说明 Memory Extractor 的判断质量比 Memory Store 的存储能力更重要。

本 Part 核心认知升级

之前对 Agent 的理解：

Agent = LLM + Tools

升级后：

Agent = LLM + Runtime + Tools + Memory

进一步拆开：

Runtime 负责：如何组织思考和行动
Tool 负责：Agent 能做什么
Memory 负责：Agent 长期知道什么

Context Builder 负责：本轮让 LLM 看到什么

Memory 的关键不是存储历史，而是保存对未来决策有价值的信息。

工业级实现

工业级 Memory 不应理解为数据库表。

错误模型：

Memory = MySQL Table

更合理的模型：

```
Memory Runtime
|
+-- Extraction
|
+-- Classification
|
+-- Deduplication
|
+-- Storage
|
+-- Retrieval
|
+-- Ranking
|
+-- Injection
|
+-- Lifecycle Management
```

Memory Extractor 的工业实现可能是：

LLM Based:

```
Conversation
|
v
LLM Extractor
|
v
Memory
```

也可能是：

Hybrid:

```
Conversation
|
v
Rule Filter
|
v
LLM Extractor
|
v
Memory Store
```

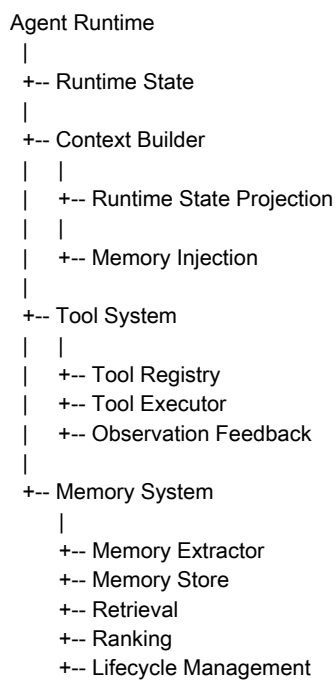
工业设计还必须考虑：

- Confidence
- Deduplication
- Conflict Resolution

- Privacy Filter
- Forget Policy
- Context Budget
- Auditability

知识地图

Day06 Part A 与前面章节的关系：



关联关系：

- Day04 Context Builder：Memory 最终要通过 Context Builder 投影给 LLM。
- Day05 Observation：Tool Result 可以成为 Memory 抽取的来源之一。
- Day05 Permission：Memory 也需要类似的 Runtime Governance。
- Day06 Memory：补齐 Agent 的长期状态系统。

面试视角

Q1：Memory 和 Conversation 有什么区别？

回答：

Conversation 是完整历史事件记录，Memory 是从历史中提取出的、未来可能影响 Agent 行为的长期有效信息。

Q2：Memory 是不是 Vector Database？

回答：

不是。Vector Database 只是 Memory Store 的一种实现。完整 Memory System 还包括 Extraction、Classification、Deduplication、Retrieval、Ranking、Injection 和 Lifecycle Management。

Q3：为什么需要 Memory Extractor？

回答：

因为不是所有历史信息都有长期价值。Memory Extractor 负责判断哪些信息应该忽略、创建、更新、合并或遗忘，并避免 Memory 污染。

Q4：Memory 和 Runtime State 有什么区别？

回答：

Runtime State 是当前任务生命周期内的运行现场，Memory 是跨会话、跨任务、长期存在的用户或业务状态。

Q5：Memory 如何进入 LLM？

回答：

Memory 不应该直接全部塞进 Prompt，而应经过 Retrieval、Ranking 和 Context Builder 的预算控制后，再被投影到当前 LLM Context。

本章思考题

为什么不能把全部 Conversation 直接作为 Memory？

如果用户说“我最近可能想学习 Go”，应该保存 Memory 吗？

Memory Conflict 应该如何处理？

Memory Retrieval 是否应该每次请求都执行？

Runtime State 和 Memory 是否可以统一设计？

Memory Decay 应该由时间驱动，还是由新证据驱动？

Tool Result 是否也应该进入 Memory Extractor？

Memory 安全策略应该放在 Extractor 前，还是 Store 前？

前置问题回收

本 Part 已经回答：

- Conversation 和 Memory 的区别
- 为什么 Agent 需要 Memory
- 为什么 Memory 需要 Extractor
- Memory 是否需要专门 LLM 判断
- Memory 为什么不是 Vector Database
- Memory 为什么类似人类记忆
- Memory 与 Runtime State 的边界
- Memory 如何通过 Context Builder 影响 LLM

待后续回答：

- Memory Store 如何设计？
- Embedding 是什么？

- Vector Database 在 Memory System 中到底扮演什么角色？
- Retrieval 如何实现？
- Ranking 为什么必要？
- Hybrid Retrieval 如何设计？
- Memory 如何控制 Context Budget？

下一节学习计划

下一节进入：

Day06 Part B : Memory Architecture (记忆系统架构)

重点拆：

- Memory Store 到底存什么
- Vector Database 为什么不是 Memory
- Embedding 如何支持语义检索
- Retrieval 为什么需要 Ranking
- Memory 如何进入 Context Builder
- ChatGPT / Claude Code 这类产品大概是什么 Memory 架构

Part B 会从“Memory 是什么”进入“Memory System 怎么设计”。